

Design and Implementation of the sclp Optimizing Compiler

CS 306: Compilers Lab — End-to-End Compiler Project Report

Group 10 (JAN2026_group_10)

Department of Computer Science and Engineering, IIT Bombay

August 16, 2026

Abstract

This report presents the architecture, intermediate representations, and target code generation pipeline of the **sclp** (Simple C-Like Program) compiler. Built iteratively across assignments A0 to A5, the compiler translates statically typed imperative source programs with integers, single-precision floats, strings, complex control structures, and multi-parameter functions into executable MIPS/SPIM assembly. We detail our modular compilation pipeline: Flex-based lexical analysis, LALR(1) Bison parsing, Abstract Syntax Tree (AST) construction with static semantic validation, Three-Address Code (TAC) lowering, Register Transfer Language (RTL) generation with linear register tracking, Activation Record (AR) layout management, and final target assembly emission.

1 Introduction & System Overview

The **sclp** compiler is a full multi-stage translation system targeting the MIPS R2000/R3000 architecture simulated via SPIM. The system enforces C-style static scoping, strong type checking, short-circuit boolean evaluation, dynamic register allocation, and standard calling conventions.

2 Compiler Architecture & Pipeline

2.1 Lexical & Syntactic Analysis

- **Lexer** (`scanner.1`): Recognizes multi-character operators (`<=`, `>=`, `==`, `!=`, `&&`, `||`), integer/float literals, identifiers, string constants, and block comments, tracking source line numbers for diagnostic reporting.
- **Parser** (`parser.y`): Built on a non-ambiguous LALR(1) grammar resolving shift-reduce conflicts in dangling-else and arithmetic precedence. It supports global variable declarations, function prototypes, local scopes, iterative (`while`, `do-while`) and selection (`if-else`) constructs.

2.2 Symbol Table & Scope Resolution

Scoped symbol resolution is governed by hierarchical tables (`Program.hpp`):

- **Scope Hierarchy**: Scopes are structured as parent-linked trees differentiating GLOBAL and FUNCTION lexical tiers.

- **Shadowing & Resolution**: Local identifiers legally shadow global symbols. Redclarations within the same scope are flagged as compile-time semantic errors.
- **Function Signatures**: Validates arity, parameter types, matching return types across all branching control paths, and forbids function overloading.

2.3 AST & Semantic Analysis

The AST (`Ast.hpp`, `Ast.cpp`) represents the parsed program as an object hierarchy rooted at `Program_AST`:

- **Type System**: Validates compatibility for `int`, `float`, `string`, and `void`. Automatic type coercion nodes (`Type_Cast_AST`) are synthesized when promoting integers to floats in mixed-mode expressions.
- **Control-Flow Checks**: Non-void functions must include explicit return statements returning compatible types; void functions are strictly barred from returning values.

2.4 Intermediate Representations: TAC & RTL

The compiler employs a two-tier Intermediate Representation:

1. **Three-Address Code (TAC)**: Decomposes complex expression trees into linearized quadruples. Generates shared temporaries (`stempN`) and conditional jump labels (`goto`, `if...goto`) implementing short-circuit boolean semantics.
2. **Register Transfer Language (RTL)**: Bridges machine-independent TAC and target MIPS instructions. Represents explicit register movements, address loads (`lw`, `l.s`), stores (`sw`, `s.s`), floating-point coprocessor conversions (`mtc1`, `cvt.s.w`), and conditional branch comparisons.

3 Memory Layout & Runtime Environment

3.1 Activation Record (Stack Frame) Architecture

Each function call creates a stack frame organized around the Frame Pointer (`$fp`) and Stack Pointer (`$sp`) as follows:

Offset / Location	Contents
Higher Addresses	Function Parameters (pushed in reverse)
0(<code>\$fp</code>)	Saved Return Address (<code>\$ra</code>)
-4(<code>\$fp</code>)	Saved Previous Frame Pointer (PFP)
-8(<code>\$fp</code>) downwards	Local Variables (order of declaration)
Below Locals	Shared Temporaries (<code>stemp_k</code>)
0(<code>\$sp</code>)	Dynamic Frame Bottom (grows down)

3.2 Function Calling Sequence (ABI)

- **Caller Prologue:** Evaluates actual parameters and pushes them onto the stack in reverse order of the formal parameter list.
- **Callee Prologue:** Allocates stack space by adjusting `$sp`, saves caller's `$ra` and `$fp`, and initializes the new frame pointer:

```
subu $sp, $sp, 8      # Space for RA and PFP
sw $ra, 4($sp)        # Save RA
sw $fp, 0($sp)        # Save Old FP
add $fp, $sp, 8       # Set New FP
subu $sp, $sp, <frame> # Allocate locals & stumps
```

- **Callee Epilogue:** Restores return value into `$v0` (integer) or `$f0` (float), resets `$sp`, restores `$fp` and `$ra`, and executes `jr $ra`.
- **Caller Cleanup:** Pops parameters off the stack and reads function return values from accumulator registers.

3.3 Register Allocation & Tracking

The `RegisterTracker` module manages physical MIPS integer registers (`$t0`–`$t9`, `$s0`–`$s7`, `$v0`, `$a0`–`$a3`) and single-precision floating-point registers (`$f0`–`$f30`). Registers are allocated on-demand with priority heuristics, spilling shared temporaries to dedicated offsets within the activation record when register pressure spikes.

4 Key Implementation Highlights

- **Mangled Function Naming:** Internal non-main functions are decorated with a trailing underscore (`fn_`) to avoid collisions with standard SPIM syscall labels and runtime primitives.
- **Deterministic Global & String Emission:** Global variables are mapped directly into the `.data` segment in declaration order. All distinct string literals are assigned global labels (`str_N`) and emitted sequentially before code generation.
- **Alphabetical Function Processing:** AST, TAC, RTL, and assembly routines are processed and emitted in deterministic alphabetical order to ensure verifiable intermediate representations.
- **Short-Circuit Logic:** Relational and boolean operators (`&&`, `||`, `!`) are lowered into control-flow jumps using specialized short-circuit labels without redundant arithmetic evaluations.

5 Verification, Testing & Results

The compiler implementation was extensively validated through an automated testing harness (`omega-tester.py`):

- **Level-by-Level Testcases:** Tested from basic arithmetic expressions (L1/L2) to control-flow structures (L3/L4) and recursive multi-function calls with mutual recursion (L5).
- **Negative Semantic Test Suite:** Validated that illegal operations (type mismatches, improper returns, uninitialized symbols, illegal casts, argument arity mismatches) are trapped with informative diagnostics and non-zero exit codes.
- **SPIM Runtime Verification:** Output assembly was compiled and executed under SPIM, matching standard

execution traces and reference outputs with 100% test suite compliance.

6 Conclusion

The `sclp` compiler project provides a robust, fully featured implementation of an optimizing ahead-of-time compiler for a C-like language. The modular separation between AST, TAC, RTL, and MIPS code emission ensures high maintainability, precise semantic validation, and efficient target code generation.